

# 스프린트 미션 16 — 딥러닝 모델 변환·양자화 요약 보고서

SimpleCNN(직접 학습) · MNIST · PyTorch(FP32) → 양자화(INT8) → ONNX | 6팀\_전민재

GitHub Repository URL

[https://github.com/minjaejeon0827/sprint\\_mission16](https://github.com/minjaejeon0827/sprint_mission16)

## 1. 미션 개요 및 접근

이전 미션에서 다른 CNN 분류 모델을 선택해 MNIST로 직접 학습한 뒤, 동일 모델을 요구된 3가지 포맷(① PyTorch 기본 .pth, ② 양자화 .pth, ③ .onnx)으로 변환·저장했다. 이후 ONNX 파일을 기반으로 세 모델을 MNIST 평가 데이터셋 전체(10,000장)로 추론·검증했다.

· 모델: SimpleCNN (Conv16→Conv32→FC10, 파라미터 대부분이 FC층) · 양자화: Static PTQ(INT8, fbgemm) · ONNX: opset 17, dynamic\_axes(배치 가변)

## 2. 추출된 모델 목록 및 용량 (한눈에 보기)

미션 16 — 변환·저장된 모델 목록 및 용량 비교

| 모델 포맷              | 파일명                           | 용량      | 용량 감소율 | 정확도(MNIST) |
|--------------------|-------------------------------|---------|--------|------------|
| PyTorch FP32 (기본)  | mission_16_simplecnn_fp32.pth | 83.4 KB | -      | 98.23%     |
| PyTorch INT8 (양자화) | mission_16_simplecnn_int8.pth | 42.6 KB | 48.9%  | 98.15%     |
| ONNX (FP32)        | mission_16_simplecnn.onnx     | 81.3 KB | 2.5%   | 98.23%     |

그림 1. 변환·저장된 3개 모델 파일과 용량·정확도. 세 파일 모두 요구된 명명 규칙 mission\_16\_{model\_name}.pth / .onnx 를 따른다.

## 3. 검증 결과 요약 (정확도 · 용량 · 속도)

| 방법                   | 정확도(%) | 용량(KB) | 추론(ms, batch=1) | P99(ms) |
|----------------------|--------|--------|-----------------|---------|
| PyTorch FP32 (.pth)  | 98.23  | 83.4   | 0.245           | 0.37    |
| PyTorch INT8 (.pth)  | 98.15  | 42.6   | 0.182           | 0.309   |
| ONNX Runtime (.onnx) | 98.23  | 81.3   | 0.043           | 0.092   |

· INT8 양자화로 용량 **48.9% 감소**(정확도 손실 0.08%p) · ONNX Runtime이 PyTorch FP32 대비 **약 5.7배** 빠름(fusion 효과, Ch.2)

· 단일 이미지 예측 일치 검증: FP32·INT8·ONNX 모두 동일 클래스 예측 → 변환 파이프라인 정상(Ch.7). 속도는 워밍업 10 + 반복 50회 측정(Ch.6).

## 4. 디버깅 · 오류 사항 및 해결 과정

### ①. 양자화 텐서에서 .view() 오류

증상 INT8 변환 모델 forward 시 **RuntimeError: view size is not compatible with input tensor's size and stride ... Use .reshape() instead.**

원인 양자화된 conv 출력 텐서의 메모리 stride가 연속(contiguous)이 아니어서, 연속성을 요구하는 .view()가 실패.

해결 flatten 연산을 `torch.flatten(x, 1)`(또는 `.reshape`)로 교체. FP32/INT8 모두에서 안전하게 동작.

## ②. 양자화 모델 저장·재로드 실패

증상 `torch.save(model, ...)`(객체 전체)로 저장한 INT8 모델을 `torch.load` 후 사용 시 `AttributeError: 'ConvReLU2d' object has no attribute '_backward_hooks'` (융합 미적용 시에도 `'Conv2d' ... _backward_hooks`).

원인 현재 PyTorch 버전에서 양자화 모듈의 pickle 역직렬화가 `nn.Module` 내부 속성을 온전히 복원하지 못하는 이슈. 또한 객체 전체 저장은 로드 시 원본 클래스 정의가 반드시 필요(`Can't get attribute 'SimpleCNN'`).

해결 양자화 모델은 `torch.jit.script(model)` → `torch.jit.save(TorchScript)`로 저장하고 `torch.jit.load`로 로드. 클래스 정의 없이도 견고하게 복원되며 배포에도 권장되는 방식.

## ③. MNIST 다운로드 실패(제한된 네트워크)

증상 `torchvision.datasets.MNIST(download=True)` 실행 시 기본 미러(S3) 접근이 막혀 `RuntimeError` 발생.

원인 실행 환경의 방화벽/프록시가 torchvision 기본 데이터 미러를 차단.

해결 로더에 fallback 추가: torchvision 실패 시 GitHub raw 미러의 `mnist.pkl.gz`를 받아 동일 정규화로 구성. Colab 등 일반 환경에서는 torchvision 경로가 그대로 사용됨.

## ④. ONNX export 방식·opset 고정

증상 최신 PyTorch에서 `torch.onnx.export` 기본 동작(dynamo) 및 opset 미지정 시 결과가 달라질 수 있음.

원인 exporter 버전·opset에 따라 그래프/재현성이 흔들릴 수 있음(Ch.2).

해결 `opset_version=17, dynamic_axes` 명시 + `dynamo=False`(미지원 버전은 예외처리로 자동 폴백). 변환 후 `onnx.checker`와 수치 동등성( $\max\_diff \approx 9.5e-07 < 1e-5$ )으로 검증.

---

## 5. 결론

요구된 3종 포맷 변환·저장과 ONNX 기반 추론 검증을 모두 완료했다. 양자화(INT8)로 용량을 약 48.9% 줄이면서 정확도 손실은 0.1%p 내외로 억제했고, ONNX Runtime은 동일 정확도를 유지하며 가장 빠른 추론을 보였다. 특히 양자화 모델의 저장·재로드에서 발생한 실제 오류를 TorchScript 직렬화로 해결한 점, 그리고 워밍업·반복 측정으로 공정하게 속도를 비교한 점이 이번 미션의 핵심 학습 포인트였다.

첨부 파일: `modeling.ipynb`, `inference.ipynb`, 3개 모델 파일(`mission_16_*.pth/.onnx`), 용량 비교 이미지.